Optimizers for Deep Learning: A Companion to Lecture 5

Every model of this course has been trained by Adam, used as given. Lecture 5 (Section 6) said in a few pages why Adam, and the newer Muon, train a Transformer far faster than gradient descent. These notes tell the whole story: each optimizer answers a specific failure of the one before it, shown on a small example.

The architecture of Lecture 4 and the data of Lecture 5 no longer matter: an optimizer sees the model through one narrow interface (Figure 1). The model has weights θ, a vector of all its P parameters. At step i, training draws a batch $x^{(i)}$ of B chunks of T tokens (a $B \times T \times d$ tensor once embedded), and the model returns two things:

1. the forward pass gives the batch's loss, $\ell^{(i)} = \mathcal{L}(\theta, x^{(i)})$, the average log loss of its next tokens;
2. the backward pass gives its gradient, $\mathbf{g}^{(i)} = \partial \ell^{(i)} / \partial \theta$, one number per parameter.

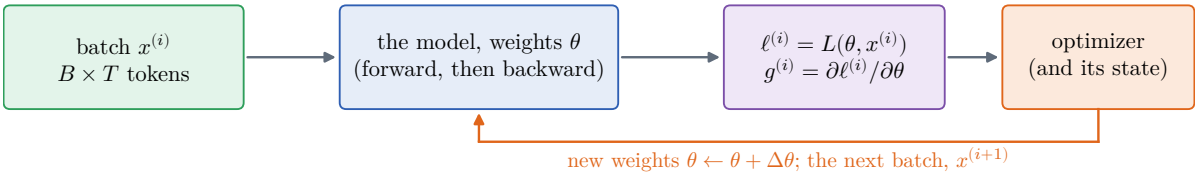


Figure 1. An optimizer sees the model only through a loss and its gradient, one batch at a time. A batch (green) goes through the model with weights θ (blue); the forward pass gives the loss, the backward pass its gradient (purple); the optimizer (orange) turns the gradient, and any state it keeps, into a change of the weights.

The optimizer turns these into a change of the weights, $\theta \leftarrow \theta + \Delta\theta$. It may also carry a few numbers of its own from one step to the next, its **state**, such as an average of past gradients. What we want to make small is the expected loss over all the text, $\mathcal{L}(\theta) = \mathbb{E}_x \mathcal{L}(\theta, x)$, of which each batch gives a noisy view. (We write $\mathcal{L}$ because, in the lectures, L counts blocks.)

Three properties make this loss hard to minimize. It is far steeper in some directions than in others, so a step small enough for the steep ones crawls along the flat ones; momentum, which remembers past gradients, builds up speed along them. Each batch's gradient is noisy, and a step size that decays over the run lets the noise average out. And different parameters' gradients differ in size by orders of magnitude, so Adam gives each parameter its own step size; inside a weight matrix, the gradient also pushes hard along a few directions and barely along the rest, which Muon evens out. All of these methods need a backward pass and memory for their state, and zeroth-order methods give up the gradient to save both. The notes have four parts:

1. one step size for everything (§§1–3): gradient descent, and why steep and flat directions slow it; stochastic gradients, and how large a batch to use; momentum and Nesterov's method;
2. a step size for each parameter (§§4–5): AdaGrad, RMSProp, Adam, and AdamW; why Adam, and not momentum, trains language models;
3. a step for each matrix (§6): the best step for a weight matrix, and Muon;
4. no gradient at all (§7): zeroth-order methods, which fine-tune a model in the memory needed to run it.

A table (§8) sums up which optimizer fixes what. The interactive optimizer explorer, built after Goh (2017), runs six of these optimizers, from gradient descent to Adam, side by side on losses of two parameters like those of §§1–4, with the step size, the gradient noise, and the starting point under your control; §§1, 2, 4, and 6 say what to try in it.

1. Gradient descent and the condition number

The simplest optimizer steps against the gradient by a fixed **step size**, or learning rate, η: $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$, where $\nabla \mathcal{L}(\theta)$ is the exact gradient of the expected loss. How large can η be? Near a minimum, any smooth loss looks like a quadratic bowl, so take the simplest one, in two parameters:

$$\mathcal{L}(\theta) = \tfrac{1}{2}(\lambda_1 \theta_1^2 + \lambda_2 \theta_2^2), \qquad \lambda_1 = 1,\ \lambda_2 = 25.$$

Each λ_j is the loss's **curvature** along θ_j, its second derivative: how fast the slope grows as θ_j moves away from 0. The gradient is $(\lambda_1 \theta_1,\ \lambda_2 \theta_2)$, so a step multiplies each coordinate by its own factor, $\theta_j \leftarrow (1 - \eta \lambda_j)\, \theta_j$.

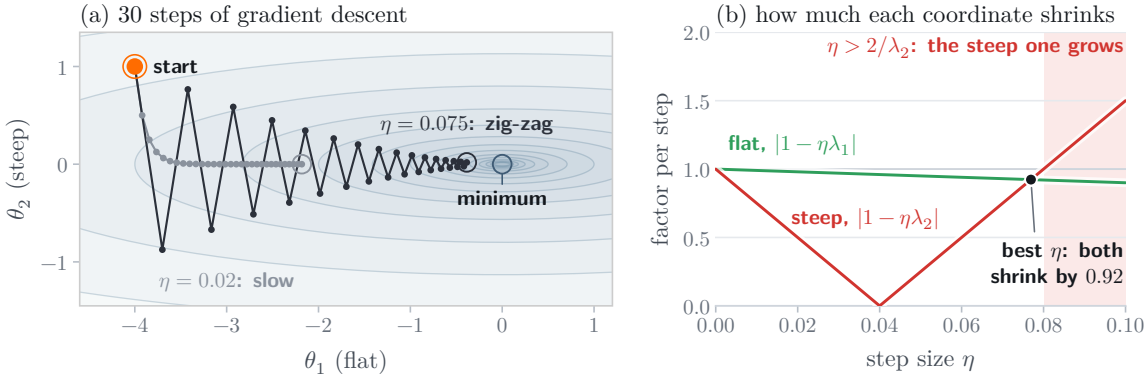


Figure 2. One step size cannot suit a flat and a steep direction. The quadratic $\frac{1}{2}(\theta_1^2 + 25\theta_2^2)$. (a) Its contours and 30 steps of gradient descent from $(-4, 1)$: at $\eta = 0.02$ the flat coordinate crawls; at $\eta = 0.075$, near both the best step (0.077) and the limit (0.08), the steep one changes sign at every step. (b) The factor by which each coordinate shrinks per step, against η.

A coordinate shrinks only if $|1 - \eta\lambda_j| < 1$, that is, $\eta < 2/\lambda_j$, so the steepest direction sets the limit for all: $\eta < 2/\lambda_{\max} = 0.08$. Near this limit the steep coordinate zig-zags, while the flat one shrinks by only $1 - \eta\lambda_{\min} \approx 0.92$ per step (Figure 2a). The slower coordinate sets the pace, so the best η makes the two factors equal in size, $1 - \eta\lambda_{\min} = \eta\lambda_{\max} - 1$ (Figure 2b). That gives $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, at which both coordinates shrink per step by

$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

The ratio κ is the **condition number**. For $\kappa = 25$ the factor is 0.92: each factor of e closer to the minimum takes about $\kappa/2 = 12.5$ steps, and a millionfold lower loss, which is a thousandfold smaller distance since the loss is quadratic, takes 87. A general quadratic, $\frac{1}{2}(\theta - \theta^*)^\top H (\theta - \theta^*)$, with its minimum at θ^* and H the matrix of its second derivatives (the Hessian), behaves the same way along each eigenvector of H, with λ_j its eigenvalues; a deep network has millions of directions, whose curvatures can differ by orders of magnitude. In the optimizer explorer (Muon is not among its optimizers: two parameters form a vector, not a matrix), the bowl $\frac{1}{2}(\theta_1^2 + \kappa\theta_2^2)$ has κ on a slider: with 300 steps, set η just below and just above $2/\kappa$, and watch the steep coordinate settle, then diverge; then raise κ, and watch the flat one stall.

What fails, and what fixes it. The flat directions crawl, because the step that keeps the steep ones stable is tiny for them. Either remember past gradients, so that consistent directions build up speed (§3), or scale the step separately for each parameter (§§4–5) and for each direction of a weight matrix (§6). First, though, the optimizer never sees the exact gradient $\nabla \mathcal{L}(\theta)$ used so far, only one batch's noisy estimate of it.

2. Stochastic gradients

The exact gradient would cost a pass over all the training text, 131 million tokens for Lecture 5's base model, so each step uses one batch's gradient: **stochastic gradient descent** (SGD). Write it as the true gradient plus noise, $\mathbf{g}^{(i)} = \nabla\mathcal{L}(\theta) + \xi^{(i)}$. Two facts about the noise matter here:

1. **It averages to zero.** The chunks are drawn at random, so $\mathbb{E}\,\mathbf{g}^{(i)} = \nabla\mathcal{L}(\theta)$.
2. **Its variance falls as $1/B$.** Let Σ be the covariance of one chunk's gradient noise, a $P \times P$ matrix whose trace $\mathrm{tr}(\Sigma)$, the sum of its diagonal, is the noise's expected squared size. The average of B independent chunks has covariance Σ/B and expected squared size $\mathrm{tr}(\Sigma)/B$: twice the batch, half the noise's variance, twice the cost.

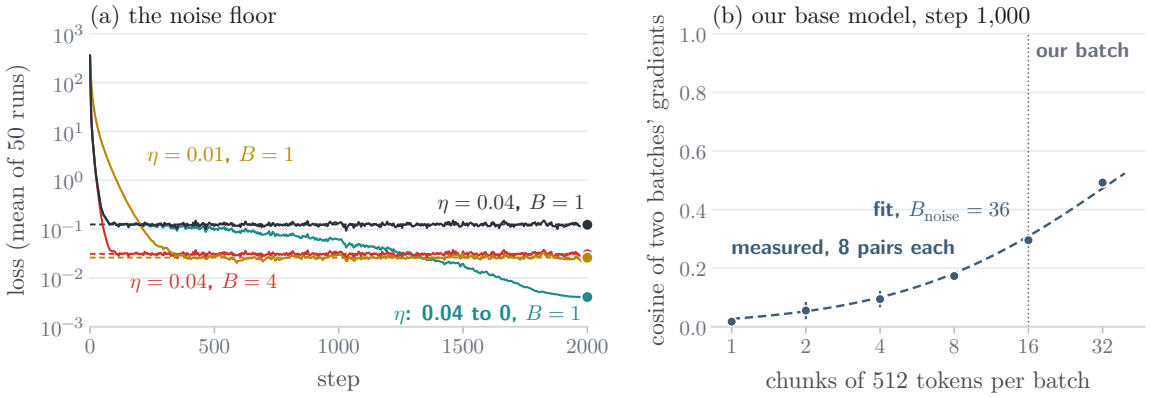


Figure 3. A fixed step size settles at a noise floor proportional to η/B; at our batch size, two batches' gradients mostly disagree. (a) SGD on a quadratic in 10 parameters with curvatures spaced geometrically from 1 to 25 and gradient noise of variance 1 per coordinate per example, from $\theta = (3, \dots, 3)$: the loss averaged over 50 runs, with the predicted floors dashed; teal, a step that decays along a cosine from 0.04 to 0. (b) The cosine of the gradients of two independent batches of Lecture 5's base model at step 1,000, against the batch's number of chunks of 512 tokens (mean and standard deviation over 8 pairs), and the fit $1/(1 + B_{\mathrm{noise}}/B)$. (a)'s loss axis and (b)'s batch axis are logarithmic.

The noise floor, and why the step size must decay

On a quadratic like that of §1, with curvatures λ_j, add noise of variance σ^2/B to each coordinate's gradient. A coordinate then follows $\theta_j \leftarrow (1 - \eta\lambda_j)\theta_j - \eta\xi_j$: shrunk as before, and kicked at every step. Its mean still shrinks to zero, but the kicks keep it spread out; the shrinking and the kicks balance when the variance s_j of θ_j stops changing, $s_j = (1 - \eta\lambda_j)^2 s_j + \eta^2\sigma^2/B$, and its share of the loss, $\frac{1}{2}\lambda_j s_j$, is then

$$\frac{\eta\,\sigma^2}{2B\,(2 - \eta\lambda_j)} \;\approx\; \frac{\eta\,\sigma^2}{4B} \qquad (\eta\lambda_j \ll 1).$$

Summed over the coordinates, whose σ^2 add up to $\mathrm{tr}(\Sigma)$, the run settles at a **noise floor** of about $\eta\,\mathrm{tr}(\Sigma)/(4B)$. The curvature has dropped out; a smaller step and a larger batch lower the floor equally; and no fixed η reaches the minimum. The measurements agree (Figure 3a): at $\eta = 0.04$ and $B = 1$ the run settles at 0.123, against 0.124 from the exact sum of the first expression (the approximation gives 0.100, since here $\eta\lambda_{\max} = 1$ is not small); a quarter of the step gives 0.026, four times slower; four times the batch gives 0.031, at four times the cost; a

decaying step reaches 0.0045. In the interactive optimizer explorer, raise the noise σ, watch each loss stop at a floor, then lower η.

Robbins and Monro (1951) gave the conditions under which steps η_i reach the minimum despite the noise, $\sum_i \eta_i = \infty$ and $\sum_i \eta_i^2 < \infty$: the steps can still travel any distance, and the noise they inject, which grows with $\sum \eta_i^2$, stays finite. A step of c/i satisfies both. Language models train for a fixed number of steps instead, holding a large step while it makes progress and decaying it toward zero at the end, the schedule of Lecture 5 (Section 7).

How large a batch?

Is a larger batch worth its price? Measure the noise. Take the gradients of two independent batches, $\mathbf{g}_a = \mathbf{G} + \xi_a$ and $\mathbf{g}_b = \mathbf{G} + \xi_b$, writing $\mathbf{G}$ for the true gradient $\nabla \mathcal{L}(\theta)$. Their dot product is about $\|\mathbf{G}\|^2$, since the noises are independent, and each has squared size about $\|\mathbf{G}\|^2 + \mathrm{tr}(\Sigma)/B$, so

$$\cos(\mathbf{g}_a, \mathbf{g}_b) \approx \frac{1}{1 + B_{\text{noise}}/B}, \qquad B_{\text{noise}} = \frac{\mathrm{tr}(\Sigma)}{\|\mathbf{G}\|^2}.$$

The cosine says how much of a step is signal. Near 1, two batches agree, and either points where the full gradient does; near 0, each points mostly in a random direction, and only the average over many steps goes downhill. B_{noise}, the **noise scale** of McCandlish et al. (2018) in its simple form, is the batch at which signal and noise are equal. On Lecture 5's base model (Figure 3b), two batches of 16 chunks (the batch it was trained with) have a cosine of 0.30, and two single chunks 0.02; the fit gives $B_{\text{noise}} \approx 36$ chunks, about 18,000 tokens. Our batch of 8,192 tokens is below it: most of each step is noise, and the run progresses because successive steps' noise cancels while their signal adds up.

Is a higher cosine then better? Per step, yes: the best step on a batch of B lowers the loss by a fraction $1/(1 + B_{\text{noise}}/B)$ of the best step on the exact gradient (McCandlish et al.). Per token, not necessarily: below a **critical batch size** of about B_{noise}, doubling the batch nearly halves the steps for nearly the same tokens; above it, doubling the batch saves few steps and nearly doubles the tokens. As the loss falls, the gradient shrinks while its noise does not, so B_{noise} grows. GPT-3's authors used the noise scale to choose its batch size, and increased the batch over its first few billion tokens (Brown et al., 2020).

Neither a decaying step nor the right batch helps with the flat directions of §1.

3. Momentum and Nesterov's method

Gradient descent forgets each step at once. On the quadratic, the flat coordinate's gradient keeps its sign and the steep one's flips at every step, so a method that remembered recent gradients could speed up along the first and cancel along the second. That is **momentum**, the "heavy ball" of Polyak (1964). It keeps a velocity $\mathbf{v}$, a running sum of gradients that decays by β per step, and steps along it:

$$\mathbf{v}_i = \beta\, \mathbf{v}_{i-1} + \mathbf{g}^{(i)}, \qquad \theta^{(i+1)} = \theta^{(i)} - \eta\, \mathbf{v}_i.$$

Without $\mathbf{v}$, $\theta^{(i+1)} = \theta^{(i)} - \eta\, \mathbf{g}^{(i)} + \beta(\theta^{(i)} - \theta^{(i-1)})$: a gradient step plus a fraction β of the previous step, the ball's inertia. Where the gradient stays the same, the velocity grows to $\mathbf{g}/(1-\beta)$, ten times gradient descent's step for the usual $\beta = 0.9$; where it alternates in sign, successive terms nearly cancel, and the velocity is only about $\mathbf{g}/2$ in size.

How much faster is it? On the quadratic of §1, put the gradient $\lambda_j \theta_j$ into the update written without $\mathbf{v}$: coordinate j, written z_i at step i, follows $z_{i+1} = (1 + \beta - \eta\lambda_j) z_i - \beta z_{i-1}$ (on a general quadratic, so does each eigen-direction). Its solutions are combinations of r^i, where

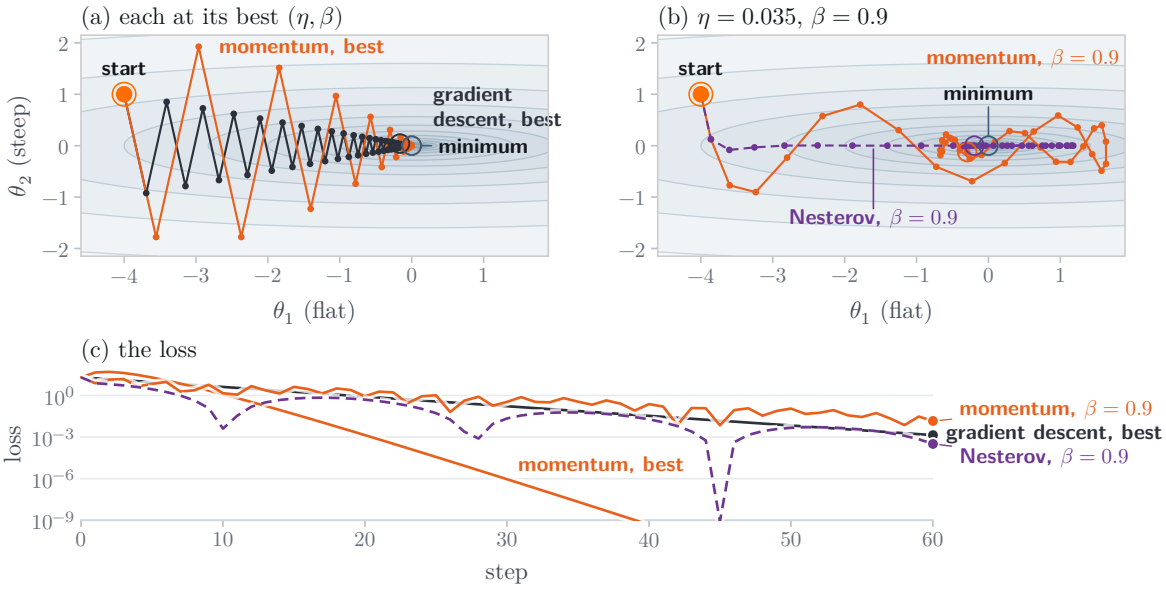


Figure 4. Momentum's rate depends on $\sqrt{\kappa}$ instead of κ; at a large β, Nesterov's look-ahead damps the steep coordinate's swing. The quadratic of Figure 2, from $(-4, 1)$. (a) Gradient descent at its best step and momentum at its best step and β: momentum swings wider at first, then shrinks by 0.67 per step instead of 0.92. (b) At $\eta = 0.035$ and the customary $\beta = 0.9$: momentum circles the minimum; Nesterov's method stops the steep coordinate's swing within a few steps. (c) The loss of the four runs, logarithmic scale.

$r^2 = (1 + \beta - \eta\lambda_j)\, r - \beta$. When the two roots are complex, the coordinate swings across zero as it shrinks, and since the roots' product is β, it shrinks by $\sqrt{\beta}$ per step, whatever its curvature. The best η and β put every coordinate in this regime, or at its edge, with β as small as possible. They can be worked out exactly (Goh, 2017):

$$\eta = \left(\frac{2}{\sqrt{\lambda_{\min}} + \sqrt{\lambda_{\max}}}\right)^2, \qquad \beta = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^2, \qquad \text{error shrinks by } \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \text{ per step.}$$

This is gradient descent's factor with $\sqrt{\kappa}$ in place of κ: for $\kappa = 25$, 0.67 instead of 0.92, and 2.5 steps per factor of e instead of 12.5; for $\kappa = 10{,}000$, 50 instead of 5,000. On our quadratic, momentum lowers the loss a millionfold in 26 steps, against 87 (Figure 4a, c).

What fails: overshooting. In practice $\beta = 0.9$, far more than this quadratic's best (0.44), because real losses have much flatter directions, and the best β grows with κ: 0.96 for $\kappa = 10{,}000$. With so much inertia the ball overshoots, and its error shrinks by only $\sqrt{\beta} \approx 0.95$ per step as it circles the minimum (Figure 4b).

The fix: look ahead. Nesterov (1983) takes the gradient where the velocity is about to carry the parameters:

$$\mathbf{v}_i = \beta\, \mathbf{v}_{i-1} + \nabla\mathcal{L}\big(\theta^{(i)} - \eta\beta\, \mathbf{v}_{i-1}\big), \qquad \theta^{(i+1)} = \theta^{(i)} - \eta\, \mathbf{v}_i.$$

If the velocity is about to overshoot, the gradient where it is heading already points back, and the step brakes. On the quadratic, the look-ahead multiplies the product of the recurrence's roots by $1 - \eta\lambda_j$, so the steep coordinate's swing shrinks by $\sqrt{\beta(1 - \eta\lambda_2)} \approx 0.34$ per step instead of 0.95. The flat coordinate still circles, at $\sqrt{\beta(1 - \eta\lambda_1)} \approx 0.93$ per step, so at the same η and β the loss stays below a millionth of its start from step 91, against 126 for momentum (the dip near step 45 in Figure 4c is a coordinate crossing zero). On smooth convex functions, Nesterov's method, with a β that grows toward 1 during the run, lowers the loss as $1/i^2$,

against $1/i$ for gradient descent; no method that uses only gradients can guarantee a better rate. Implementations take the gradient at the current point and step along $\mathbf{g}^{(i)} + \beta\mathbf{v}_i$ instead of $\mathbf{v}_i$ (a rewriting of Bengio et al., 2013, equivalent to the look-ahead form of Sutskever et al., 2013), the form Muon uses (§6).

Momentum and noise. The velocity averages about $1/(1-\beta)$ gradients, which looks as if it should average away the noise of §2. But the step is also $1/(1-\beta)$ times larger, and the two cancel: at small step sizes, momentum behaves like SGD at the step $\eta/(1-\beta)$, floor included. On the noisy quadratic of Figure 3a, momentum at $\eta = 0.004$, $\beta = 0.9$ settles at 0.099, close to SGD's 0.123 at $\eta = 0.04$. Momentum helps with the curvature, not with the noise.

What still fails. Momentum still moves every parameter with the same η and speeds them all up alike. In a Transformer, the gradients of different parameters differ in size by four orders of magnitude, and with one step size the parameters with the smallest gradients barely move (§4).

4. A step size for each parameter

How different are a Transformer's gradients? For each parameter of Lecture 5's base model at step 1,000, take the root mean square of its gradient over eight batches (Figure 5a). These sizes range from 7.7×10^{-8} for the smallest 1% of the parameters to 1.5×10^{-3} for the largest 1%. The smallest are in the token table: a rare token's row is read only in the few batches where the token occurs, and as an output it is pushed down only in proportion to its small probability. With one step size, those rows move about 20,000 times less per step than the parameters with the largest gradients. The fix is a step size per parameter, scaled by its own gradients.

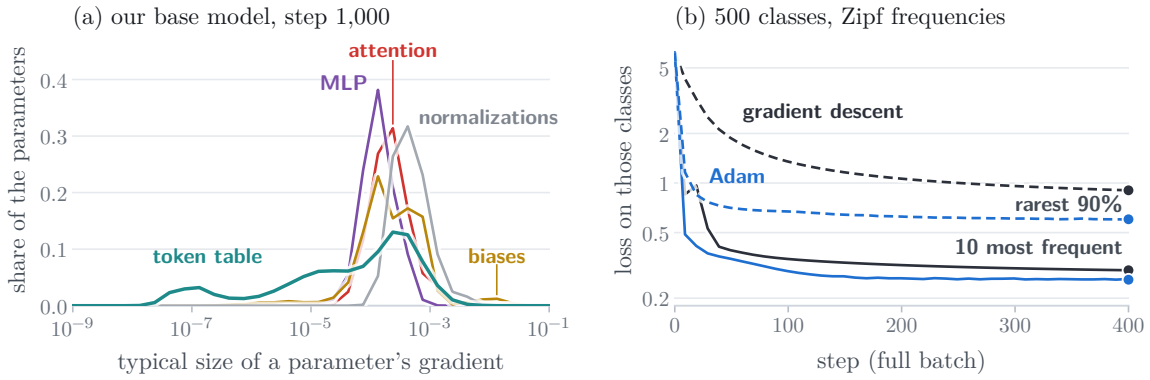


Figure 5. A language model's gradients differ in size by orders of magnitude, and gradient descent learns rare classes slowly. (a) For each parameter of Lecture 5's base model at step 1,000, the root mean square of its gradient over eight batches; the distribution of these sizes for each kind of parameter. (b) A linear softmax classifier on 500 classes whose frequencies follow Zipf's law (§5), trained on the full batch by gradient descent (black) and Adam (blue), each at the best of several step sizes: the loss on the 10 most frequent classes (solid) and on the rarest 90% (dashed), which hold 34% of the examples; the loss on a logarithmic scale.

AdaGrad (Duchi et al., 2011) divides each parameter's step by the root of the sum of its squared gradients so far. For parameter j at step i:

$$G_{i,j} = \sum_{i' \le i} \left(g_j^{(i')}\right)^2, \qquad \theta_j \leftarrow \theta_j - \eta \frac{g_j^{(i)}}{\sqrt{G_{i,j}} + \epsilon},$$

with a small ϵ against division by zero. A parameter with large gradients gets a small step size, and one whose gradient is usually zero, such as a rare token's row, keeps a large step size for the

few times it is used. **What fails:** G only grows. With gradients of constant size it grows like i, and the step shrinks like $\eta/\sqrt{i}$, a tenth of η by step 100, near the minimum or not (Figure 6a).

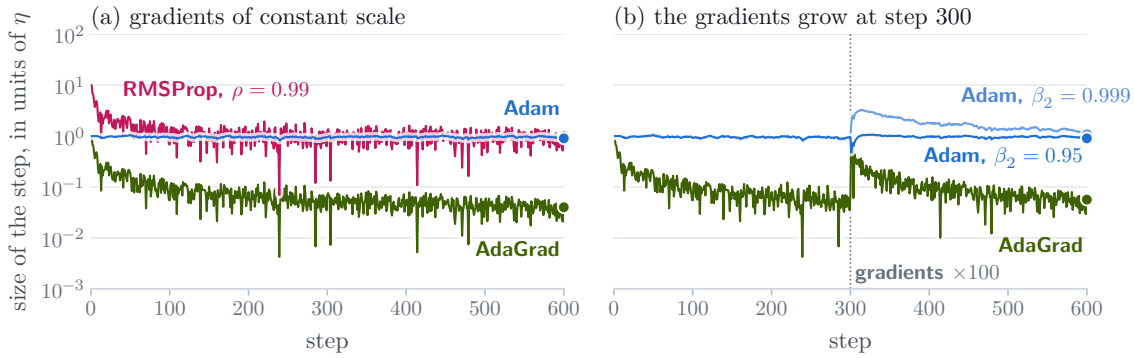


Figure 6. AdaGrad's steps shrink forever and RMSProp's start too large; Adam's stay near η, and with a short memory of squared gradients they recover fast when the gradients jump. The size of one parameter's step, in units of η, for gradients of constant mean with noise of 30% of the mean; logarithmic scale. (a) AdaGrad (olive), RMSProp (magenta, $\rho = 0.99$), and Adam (blue, $\beta = (0.9, 0.95)$). (b) The gradients grow a hundredfold at step 300: Adam with $\beta_2 = 0.999$ (light blue) takes steps of up to 3.3η for hundreds of steps; with $\beta_2 = 0.95$ (blue), it is back near η within 6 steps.

RMSProp (Tieleman and Hinton, 2012) replaces the sum by a moving average that forgets:

$$s_{i,j} = \rho\, s_{i-1,j} + (1-\rho)\big(g_j^{(i)}\big)^2, \qquad \theta_j \leftarrow \theta_j - \eta\, \frac{g_j^{(i)}}{\sqrt{s_{i,j}} + \epsilon},$$

with ρ usually 0.9 or 0.99. The average covers about the last $1/(1-\rho)$ steps, so the step stays near η for the whole run. **What fails:** the average starts at zero, so the first step is $\eta/\sqrt{1-\rho}$, ten times η for $\rho = 0.99$, and each step follows one batch's noisy gradient (Figure 6a). In the interactive optimizer explorer, choose "gradients of very different sizes", $\frac{1}{2}(10\theta_1^2 + \theta_2^2/10)$: gradient descent must keep η below 0.2 and barely moves θ_2; turn on AdaGrad and RMSProp, and watch AdaGrad's steps shrink while RMSProp's do not.

Adam (Kingma and Ba, 2015) fixes both of RMSProp's failures: it keeps an average of the gradients, as momentum does, beside RMSProp's average of their squares, and corrects both averages for starting at zero. For each parameter, at step i:

1. Update the averages: $m_i = \beta_1 m_{i-1} + (1-\beta_1) g^{(i)}$ and $v_i = \beta_2 v_{i-1} + (1-\beta_2)\big(g^{(i)}\big)^2$.
2. Correct their bias: $\hat{m}_i = m_i/(1-\beta_1^i)$ and $\hat{v}_i = v_i/(1-\beta_2^i)$.
3. Step: $\theta \leftarrow \theta - \eta\, \hat{m}_i/(\sqrt{\hat{v}_i} + \epsilon)$.

Why step 2? Unrolled, $m_i = (1-\beta_1)\sum_{i' \le i} \beta_1^{i-i'} g^{(i')}$; for gradients of constant mean, $\mathbb{E}\, m_i = (1-\beta_1^i)\,\mathbb{E}\, g$, so the division removes the bias toward zero, and likewise for v_i. The step is η times the ratio of the average gradient to the gradient's typical size (its root mean square), so it is about η long whatever the scale of the gradients: gradients of 0.01 and 2 give first steps of exactly η each, where gradient descent's would be 0.01η and 2η. Where the gradient keeps changing sign, its average is small while its typical size is not, and the step is shorter. Each setting has a reason:

1. $\beta_1 = 0.9$ averages the gradients over about the last 10 steps, like momentum.
2. $\beta_2 = 0.95$ **in language models, 0.999 by default.** When the gradients suddenly grow, as they can before a loss spike, an average over 1,000 steps still remembers the small ones, and the step is several times η for hundreds of steps; an average over 20 steps catches up within a few (Figure 6b). GPT-3 used 0.95, as have most language models since.

3. $\epsilon = 10^{-8}$ is the gradient size below which Adam stops normalizing and behaves like gradient descent with the step η/ϵ. Our smallest gradients are only about eight times larger, so ϵ shortens their steps by about a tenth.

AdamW. Weight decay shrinks every weight toward zero at each step. As **L2 regularization**, a term $\frac{\lambda}{2}\|\theta\|^2$ in the loss (this λ is the decay rate, not a curvature), it adds $\lambda\theta$ to the gradient. For gradient descent the two forms are the same: the step $-\eta(\mathbf{g} + \lambda\theta)$ shrinks every weight by the factor $1 - \eta\lambda$ besides following the gradient. Inside Adam it is not (Loshchilov and Hutter, 2019): the added $\lambda\theta$ is divided by $\sqrt{\hat{v}}$ with the rest of the gradient, so a weight whose gradients are typically 10^{-3} is decayed a thousand times harder than one whose gradients are typically 1. AdamW applies the decay apart from the normalized step,

$$\theta \leftarrow \theta - \eta\Big(\frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} + \lambda\theta\Big),$$

so every weight shrinks by the same factor $1 - \eta\lambda$ per step (Lecture 5 (Section 7) says which tensors are decayed).

5. Why Adam, and not momentum, trains language models

On a quadratic, momentum beats Adam, which at a fixed η keeps taking steps of about η and circles the minimum. On language models it is the other way around, by far.

Lecture 5 compared five optimizers on a small model, Lecture 4's model shape with Lecture 5's vocabulary of 4,096 tokens, trained for 800 steps of 16×256 tokens (Lecture 5, Fig. 22). At each optimizer's best rate, SGD reached a validation loss of 4.22 nats per token, momentum 3.51, Adam 2.85, and AdamW 2.84; Muon (§6) did better still. These are single runs on a small model, but the ranking matches practice: large language models are trained with Adam's family (AdamW, or Adafactor, which saves memory) or, lately, Muon. Two findings point to causes of the gap, and a third rules out the obvious suspect:

1. **Groups of parameters with very different curvature.** In a Transformer, the curvature of the loss differs greatly from one group of parameters to another (the token table, each attention matrix, each MLP), unlike in a convolutional network, and one step size for all falls behind when the groups differ (Zhang et al., 2024, who call the groups "blocks"). Figure 5a shows a similar spread across our base model's kinds of parameters, in the sizes of their gradients rather than in their curvature.
2. **Heavy-tailed word frequencies.** Words follow Zipf's law: the k-th most frequent word occurs about $1/k$ times as often as the most frequent one, so a large share of a text's tokens belong to words that are individually rare. The gradient that teaches a token is proportional to how often it occurs, so gradient descent lowers the loss on rare tokens slowly. Adam does not depend on frequency this way, and neither does **sign descent**, which moves every parameter by η against the sign of its gradient (Kunstner et al., 2024). Figure 5b reproduces this with no noise at all: after 400 full-batch steps, gradient descent's loss is 0.30 on the 10 most frequent classes and 0.90 on the rarest 90%; Adam's is 0.26 and 0.60, about the same on the frequent classes and a third lower on the rare ones.
3. **Noise is not the main reason.** Kunstner et al. (2023) found that the gap between gradient descent and Adam on Transformers remains, and grows, when the batch is so large that the noise is negligible, and that sign descent with momentum closes much of it. Noise does add to the gap when it is heavy-tailed, with rare gradients far larger than typical ones, which clipping (Lecture 5, §7) and adaptive steps tame (Zhang et al., 2020).

A language model's difficulty is less one steep direction against one flat one than millions of parameters whose gradients differ in scale. A step size per parameter answers that, but not the uneven directions inside a weight matrix.

6. A step for each matrix: Muon

Most of a Transformer's parameters sit in weight matrices, whose gradients have a structure that one number at a time cannot see. Write the gradient G of a $d_{\text{in}} \times d_{\text{out}}$ matrix by its singular value decomposition, $G = USV^\top = \sum_k s_k \mathbf{u}_k \mathbf{v}_k^\top$, where the columns $\mathbf{u}_k$ of U and $\mathbf{v}_k$ of V are orthonormal and the singular values $s_k \geq 0$ sit on the diagonal of S. Each term pairs an input direction $\mathbf{u}_k$ with an output direction $\mathbf{v}_k$, and s_k says how hard the gradient pushes along that pair. In Lecture 5's base model at step 1,000, the singular values of the first block's query gradient span a factor of about 40,000, and its ten largest of 192 carry 73% of its squared size (Figure 7c). A gradient step moves the matrix far along a few directions and hardly at all along the rest. Each of those directions mixes many parameters, so dividing parameter by parameter cannot even them out, and it can make them more uneven. Take a 2×2 gradient with rows $(2, 1)$ and $(1, 2)$. It pushes three times harder along $(1, 1)$ than along $(1, -1)$: its singular values are 3 and 1. On a gradient that stays the same, Adam moves every entry by η (§4), so its step has all four entries equal: it moves along $(1, 1)$ alone, and loses the weaker direction entirely. The interactive optimizer explorer shows the same failure on the bowl of §1 rotated by 45°.

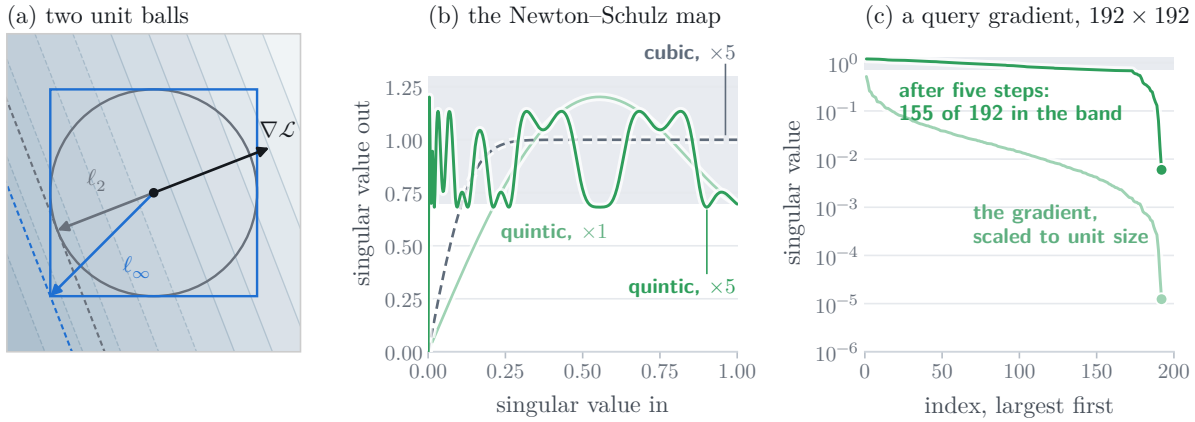


Figure 7. Muon steps along the gradient's directions with every singular value set to about 1. (a) The best step of size 1 against the gradient (black), over the first-order change of the loss, $\langle \nabla\mathcal{L}, \Delta\theta \rangle$, shaded darker where it is lower: in the Euclidean norm (gray circle) the normalized gradient; in the max norm (blue square) the corner $-\text{sign}(\nabla\mathcal{L})$, which reaches a lower level (dashed lines). (b) What Muon's iteration does to a singular value: Jordan et al.'s quintic $as + bs^3 + cs^5$ once (light) and five times (dark), and the classic cubic $\frac{3}{2}s - \frac{1}{2}s^3$ five times (dashed). (c) The singular values of our base model's first query gradient at step 1,000, largest first, scaled to unit size (light) and after five steps of the iteration (dark), with 0.7 to 1.3 shaded; logarithmic scale.

Steepest descent in a norm

Which step is best? For a small change ΔW, the loss changes by about $\langle G, \Delta W \rangle = \sum_{ab} G_{ab} \Delta W_{ab}$, the first-order term of its Taylor expansion. The best step makes this most negative among changes of a given size η, and the answer depends on how size is measured (Bernstein and Newhouse, 2024):

1. **Euclidean size**, the root of the sum of squared entries: $\Delta W = -\eta\, G/\|G\|_F$, the gradient's direction (Figure 7a).
2. **The largest entry** (the max norm): $\Delta W = -\eta\, \mathrm{sign}(G)$, every entry moved by η (Figure 7a). This is Adam with $\beta_1 = \beta_2 = 0$, since $\hat{m}/\sqrt{\hat{v}}$ is then $g/|g|$.
3. **The spectral norm**, the largest singular value: $\Delta W = -\eta\, UV^\top$, the gradient's directions with every singular value set to 1. For the 2×2 gradient above, $UV^\top$ is the identity, which pushes equally along $(1, 1)$ and $(1, -1)$.

Why measure a matrix's step this way? A layer maps an input $\mathbf{x}$ to $\mathbf{x}W$, and the step changes that output by $\mathbf{x}\Delta W$, of size at most $\|\mathbf{x}\|$ times the spectral norm of ΔW: the spectral norm is the most a step can change any input's output, and $UV^\top$ spends that budget fully in every direction instead of mostly in a few.

The third step takes two lines to check. If ΔW has spectral norm at most η, it stretches no unit vector by more than η, so for the unit vectors $\mathbf{u}_k$ and $\mathbf{v}_k$, each $\mathbf{u}_k^\top \Delta W \mathbf{v}_k$ is at least $-\eta$, so $\langle G, \Delta W \rangle = \sum_k s_k\, \mathbf{u}_k^\top \Delta W \mathbf{v}_k \geq -\eta \sum_k s_k$, and $-\eta UV^\top$ reaches the bound.

Muon

Muon (Jordan et al., 2024) adds momentum and approximates $UV^\top$ without a singular value decomposition. For each weight matrix W inside the blocks, at each step:

1. Update the momentum, $M \leftarrow \beta M + G$, with $\beta = 0.95$, and take its Nesterov form, $N = G + \beta M$ (§3).
2. Scale it to unit size, $X \leftarrow N/\|N\|_F$, so that every singular value lies in $[0, 1]$.
3. Five times, $X \leftarrow aX + b\,(XX^\top)X + c\,(XX^\top)^2 X$, with $(a, b, c) = (3.4445,\ -4.7750,\ 2.0315)$.
4. Step: $W \leftarrow W - \eta \sqrt{\max(1,\, d_{\text{out}}/d_{\text{in}})}\; X$. The factor enlarges the step of a matrix with more outputs than inputs, such as an MLP's first matrix, so that its entries are as large as those of a square matrix with the same inputs, about $\eta/\sqrt{d_{\text{in}}}$.

Since $(XX^\top)X = US^3V^\top$, step 3 leaves the directions U and V alone and replaces every singular value s by $p(s) = as + bs^3 + cs^5$. The classic **Newton–Schulz** cubic, $\frac{3}{2}s - \frac{1}{2}s^3$, applied again and again, carries every s in $(0, 1]$ to 1, but slowly from small values: its slope at zero is 1.5, so five steps lift a small singular value only $1.5^5 \approx 7.6$-fold (Figure 7b). Jordan et al. chose a quintic with slope 3.4445 at zero, whose five steps lift a small value about 480-fold, at the price of landing singular values anywhere in about [0.7, 1.3] instead of at 1, which they found does not hurt training. After five steps of the quintic, 155 of our query gradient's 192 singular values lie in that range, against 15 after five steps of the cubic; the other 37 are the smallest, still below 0.7 (Figure 7c). In large runs the iteration costs under 1% of a step's computation; for our base model, which reads only 8,192 tokens per step, it would cost about a tenth. The token table, the output layer (in our model, the same matrix), and the vectors, such as the biases and the normalizations' scales and shifts, keep AdamW.

The evidence. In a public competition to train a small GPT-2 to a target loss as fast as possible, Muon cut the training time by a factor of about 1.35 (Jordan et al., 2024). Liu et al. (2025) added weight decay to Muon and found that it matched AdamW with about 52% of the computation, and trained Moonlight, a model of 16 billion parameters, on 5.7 trillion tokens. Kimi K2, about a trillion parameters, was trained on 15.5 trillion tokens with a variant of Muon, "with zero loss spike" (Kimi Team, 2025). On our model, Muon reached 2.53 against Adam's 2.85 (Lecture 5, Fig. 22). Why evening out the directions helps is not settled; Jordan et al. speculate that the small ones are "rare directions" that matter for learning.

7. No gradient at all: zeroth-order methods

Every method so far needs a gradient and keeps state, and both cost memory that running the model does not. The backward pass stores the forward pass's activations, and AdamW in mixed precision keeps about 16 bytes per parameter (the weights, their gradient, a 32-bit copy, m, and v), where running the model takes 2: for 30 billion parameters, 480 GB against 60 GB. **Zeroth-order** methods estimate the gradient from values of the loss alone, with forward passes only.

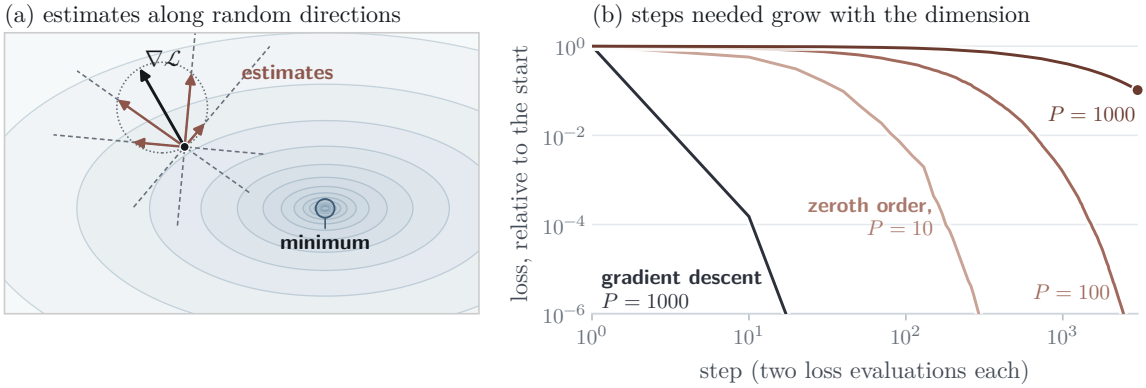


Figure 8. A zeroth-order estimate is the slope along a random direction: right on average, but its steps must be about P times smaller. (a) A loss in two parameters and its gradient $\mathbf{g}$ at a point (black). Along each of four unit directions $\mathbf{z}$ (dashed), the projection $(\mathbf{z}^\top\mathbf{g})\,\mathbf{z}$ (brown) ends on the circle that has the gradient as its diameter. For unit directions these projections average $\mathbf{g}/P$, so the estimate scales them by P; with Gaussian $\mathbf{z}$, as in the estimate below, they average $\mathbf{g}$ as they are. (b) On quadratics in P parameters, the loss relative to the start against the number of steps, for gradient descent ($P = 1000$) and for the estimate with Gaussian $\mathbf{z}$, two loss evaluations per step, at a step $1/(P+2)$ times gradient descent's (mean of 5 runs). Logarithmic scales.

Pick a random direction $\mathbf{z}$, one standard normal number per parameter. Two forward passes give the slope of the loss along it, and the estimate is the slope times the direction (Spall, 1992):

$$\hat{\mathbf{g}} = \frac{\mathcal{L}(\theta + \varepsilon\mathbf{z},\, x) - \mathcal{L}(\theta - \varepsilon\mathbf{z},\, x)}{2\varepsilon}\,\mathbf{z} \;\approx\; (\mathbf{z}^\top\mathbf{g})\,\mathbf{z}.$$

It is right on average, $\mathbb{E}\,(\mathbf{z}^\top\mathbf{g})\mathbf{z} = \mathbb{E}\,(\mathbf{z}\mathbf{z}^\top)\,\mathbf{g} = \mathbf{g}$, since $\mathbb{E}\,\mathbf{z}\mathbf{z}^\top$ is the identity. But any one estimate points along $\mathbf{z}$ (Figure 8a), and its squared size is about $P + 2$ times the gradient's. All but about $1/P$ of that is noise, a random kick that the curvature turns into loss, as in §2. So a stable step must be about P times smaller, and a given loss takes about P times as many steps as gradient descent (Figure 8b): for our base model's 3.5 million parameters, millions of times as many, which rules out pretraining.

MeZO (Malladi et al., 2023) fits the estimate into the memory of inference. Since $\mathbf{z}$ is fixed by the seed of a random number generator, it is never stored:

1. Draw a seed; add $\varepsilon\mathbf{z}$ to the weights in place, and compute the loss on a batch.
2. Regenerate $\mathbf{z}$, subtract $2\varepsilon\mathbf{z}$, and compute the loss again.
3. Regenerate $\mathbf{z}$ once more, and add $(\varepsilon - \eta \times \text{slope})\,\mathbf{z}$: this restores the weights and takes the step.

On one 80 GB A100, MeZO fine-tuned a model of 30 billion parameters, where backpropagation fit only 2.7 billion; with OPT-13B it matched full fine-tuning on 7 of 11 tasks with about a twelfth of the memory. The price is steps: 100,000 to fine-tune RoBERTa-large, of 355 million

parameters, against 1,000 with backpropagation. That is a hundred times more, not P times. The slowdown depends on the curvature's effective rank, roughly the number of directions along which the loss curves sharply, and that number is small when a pretrained model is fine-tuned with a good prompt: MeZO "only works when using prompts".

Evolution strategies use the same idea with many directions and a reward instead of a loss (Salimans et al., 2017). Draw N random directions $\mathbf{z}_n$ as above, score each perturbed model $\theta + \varepsilon\mathbf{z}_n$ with a reward R_n, and move toward the better ones, $\theta \leftarrow \theta + \frac{\eta}{N\varepsilon}\sum_n R_n\mathbf{z}_n$; the N perturbed models are the **population**. The reward can be whether an answer is right, with no gradient at all. Qiu et al. (2025) fine-tuned all the parameters of language models of up to 8 billion parameters this way, with a population of only 30. On the arithmetic puzzle Countdown they beat the reinforcement-learning methods PPO and GRPO (Weeks 6 and 7) on every model they tried; on Qwen2.5-7B, 66.8% against 57.5% for Dr. GRPO, the best of the reinforcement-learning baselines. Zeroth-order methods pay in steps to save memory, and give up the gradient to optimize any reward. They suit fine-tuning a model that nearly does the task, which is also the setting of reinforcement learning, and not pretraining.

8. Which optimizer when

Each method up to Muon fixed a failure of the one before it; zeroth-order methods instead give up the gradient to save memory:

optimizer	its step	state per parameter	fixes	still fails when
gradient descent (§1)	$-\eta\mathbf{g}$	none		curvatures differ (κ steps)
SGD, decaying η (§2)	$-\eta_i\mathbf{g}^{(i)}$	none	the noise floor	as above
momentum, Nesterov (§3)	$-\eta\mathbf{v}$	1	κ becomes $\sqrt{\kappa}$	gradient scales differ
AdaGrad (§4)	$-\eta\mathbf{g}/\sqrt{\sum\mathbf{g}^2}$	1	scales differ	steps shrink forever
RMSProp (§4)	$-\eta\mathbf{g}/\sqrt{\text{avg}\,\mathbf{g}^2}$	1	AdaGrad's shrinking	noisy, too large at first
Adam, AdamW (§4)	$-\eta\hat{\mathbf{m}}/\sqrt{\hat{\mathbf{v}}}$	2	RMSProp's noise and first steps; decay kept apart	a matrix's directions
Muon (§6)	$-\eta\,UV^\top$ of the momentum	1 (matrices only)	a matrix's directions	applied to tables and vectors
zeroth order (§7)	slope along $\mathbf{z}$, times $\mathbf{z}$	none	memory; any reward	about P times the steps

In 2026, a typical language model is pretrained with AdamW ($\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 on the matrices), a warmup, and a decay to a tenth of the peak or less, or with Muon for the matrices inside the blocks and AdamW for the rest.

References

- **Herbert Robbins and Sutton Monro (1951), A Stochastic Approximation Method.** Paper.
- **Boris T. Polyak (1964), Some Methods of Speeding up the Convergence of Iteration Methods.** Paper.
- **Yurii Nesterov (1983), A Method of Solving a Convex Programming Problem with Convergence Rate O(1/k²).** Paper.
- **Yoshua Bengio, Nicolas Boulanger-Lewandowski, and Razvan Pascanu (2013), Advances in Optimizing Recurrent Networks.** Paper.
- **Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton (2013), On the Importance of Initialization and Momentum in Deep Learning.** Paper.
- **Gabriel Goh (2017), Why Momentum Really Works.** Article.
- **Sam McCandlish, Jared Kaplan, Dario Amodei, and the OpenAI Dota Team (2018), An Empirical Model of Large-Batch Training.** Paper.
- **Tom B. Brown et al. (2020), Language Models are Few-Shot Learners.** Paper.
- **John Duchi, Elad Hazan, and Yoram Singer (2011), Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.** Paper.
- **Tijmen Tieleman and Geoffrey Hinton (2012), Lecture 6.5: Divide the Gradient by a Running Average of Its Recent Magnitude.** *Neural Networks for Machine Learning*, Coursera. Slides.
- **Diederik P. Kingma and Jimmy Ba (2015), Adam: A Method for Stochastic Optimization.** Paper.
- **Ilya Loshchilov and Frank Hutter (2019), Decoupled Weight Decay Regularization.** Paper.
- **Jingzhao Zhang, Sai Praneeth Karimireddy, Andreas Veit, Seungyeon Kim, Sashank J. Reddi, et al. (2020), Why are Adaptive Methods Good for Attention Models?** Paper.
- **Frederik Kunstner, Jacques Chen, Jonathan Wilder Lavington, and Mark Schmidt (2023), Noise Is Not the Main Factor Behind the Gap Between SGD and Adam on Transformers, but Sign Descent Might Be.** Paper.
- **Frederik Kunstner, Robin Yadav, Alan Milligan, Mark Schmidt, and Alberto Bietti (2024), Heavy-Tailed Class Imbalance and Why Adam Outperforms Gradient Descent on Language Models.** Paper.
- **Yushun Zhang, Congliang Chen, Tian Ding, Ziniu Li, Ruoyu Sun, et al. (2024), Why Transformers Need Adam: A Hessian Perspective.** Paper.
- **Jeremy Bernstein and Laker Newhouse (2024), Old Optimizer, New Norm: An Anthology.** Paper.
- **Keller Jordan et al. (2024), Muon: An Optimizer for Hidden Layers in Neural Networks.** Post.
- **Jingyuan Liu et al. (2025), Muon is Scalable for LLM Training.** Paper.
- **Kimi Team (2025), Kimi K2: Open Agentic Intelligence.** Paper.
- **James C. Spall (1992), Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation.** Paper.
- **Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever (2017), Evolution Strategies as a Scalable Alternative to Reinforcement Learning.** Paper.

- **Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D. Lee, et al. (2023), Fine-Tuning Language Models with Just Forward Passes.** Paper.
- **Xin Qiu, Yulu Gan, Conor F. Hayes, Qiyao Liang, Yinggan Xu, et al. (2025), Evolution Strategies at Scale: LLM Fine-Tuning Beyond Reinforcement Learning.** Paper.
- **Bidipta Sarkar, Mattie Fellows, Juan Agustin Duque, et al. (2025), Evolution Strategies at the Hyperscale.** Paper.